

Project Report

Assessment date: 2026-09-28

Repository: <https://github.com/Pankajkumar510/fakeageanalysis>

Live deployment: <https://fakeageanalysis-production.up.railway.app>

Executive Summary

FakeAgeAnalysis is a prototype video-analysis system with three independent machine-learning capabilities:

1. **Age estimation** from sampled video frames using a ResNet18 regression model.
2. **Deepfake classification** from sampled video frames using a ResNet18 binary classifier.
3. **Remote photoplethysmography (rPPG)** heart-rate estimation from a green-channel video signal using a small 1D convolutional regressor.

In practical terms, the project analyzes an uploaded video for whether it appears fake, estimates the subject's age, and provides a non-diagnostic health analysis based on estimated heart rate.

The application exposes a FastAPI service in `api.py` and serves the static frontend from `frontend/index.html`. Training is implemented in `train_age_deepfake.py` and `train_rppg.py`; evaluation is implemented in `evaluate_models.py`.

The project is suitable as an academic or proof-of-concept demonstration. It is not production ready yet. The largest risks are the very small rPPG dataset, weak rPPG validation design, lack of face detection and tracking, frame-level rather than video-level modeling, incomplete robustness testing, and deployment constraints for PyTorch/OpenCV inference.

Implemented Architecture

API and application flow

1. A user uploads a video to `POST /api/analyze`.
2. The API stores the upload temporarily in `temp_uploads/`.
3. Age and deepfake models process sampled video frames.
4. The rPPG model processes a normalized green-channel signal extracted from a broad central region of each frame.
5. The API returns age, deepfake label and probability, heart rate, and a basic health status.
6. The temporary file is deleted in a `finally` block.

The frontend is a static Vue 3 page loaded from a CDN. The API currently allows all CORS origins, which is convenient for development but unsafe as a production default.

Model implementations

- **Age:** ResNet18 with a one-value regression head and Smooth L1 loss.
- **Deepfake:** ResNet18 with a two-class head, weighted cross-entropy, label smoothing, and early stopping on validation accuracy.
- **rPPG:** Three 1D convolution layers followed by global average pooling and a scalar heart-rate regression head.
- **Shared image preprocessing:** Resize to 224 x 224, convert to tensor, and ImageNet normalization.
- **Training:** AdamW, ReduceLROnPlateau for age/deepfake, deterministic seeds, and gradient clipping for image models.

Technologies Used

Backend and API

- **Python 3.11:** application, training, and evaluation language.
- **FastAPI:** asynchronous HTTP API and video-upload endpoint.
- **Uvicorn:** ASGI server used to run the FastAPI application.
- **Pydantic/FastAPI validation:** request and response handling through FastAPI.

Machine Learning and data processing

- **PyTorch:** model definition, training, checkpoint loading, and inference.
- **Torchvision:** ResNet18 architecture, image transforms, and pretrained ImageNet weights.
- **NumPy:** signal normalization, statistics, and numerical array operations.
- **Pillow:** image loading and RGB conversion.
- **OpenCV:** video decoding, frame sampling, region extraction, and green-channel signal extraction.
- **scikit-learn:** optional evaluation dependency for accuracy, precision, recall, F1, and MAE metrics.
- **tqdm:** training progress reporting for the rPPG pipeline.

Frontend and deployment

- **HTML, CSS, and JavaScript:** browser application structure and interactions.
- **Vue 3 via CDN:** frontend state management and UI behavior.
- **Tailwind CSS via CDN:** frontend styling and responsive layout utilities.
- **Docker:** reproducible CPU inference environment with PyTorch and OpenCV system libraries.
- **Railway:** current public hosting platform for the combined API and frontend service.

- **GitHub:** source control and deployment source repository.

Dataset Inventory

Task	Training data	Validation data	Test data
Age estimation	6,106 rows	1,526 rows	Not separately evaluated
Deepfake detection	16,000 images	2,000 images	2,000 images
rPPG heart rate	Derived from 10 videos	Derived from the same small collection	No independent test set

The age training code now excludes labels outside 0-100. The rPPG code recognizes both `empatica_e4` and `empatica_data` sensor-folder names and aligns the BVP target window to the duration of the video signal window.

Measured Model Efficiency

The following values are the recorded results in `models/evaluation_metrics.json`. They are baseline measurements from the available checkpoint and should be regenerated after every retraining run.

Model	Samples evaluated	Metric	Recorded result
Age	1,526	MAE	5.31 years
Age	1,526	Within 5 years	14.98%
Deepfake	2,000	Accuracy	89.05%
Deepfake	2,000	Precision	91.59%
Deepfake	2,000	Recall	86.00%

Model	Samples evaluated	Metric	Recorded result
Deepfake	2,000	F1	88.71%
rPPG	4	MAE	74.66 BPM

Efficiency and operational observations

- ResNet18 is relatively small compared with modern video transformers, but the API runs CPU inference frame by frame and loads three models at process startup.
- The API does not batch sampled frames, so video inference can be slower than necessary.
- The frontend and API upload the full video to local disk. There is no file-size, duration, codec, or content validation.
- Model files are approximately 44.8 MB each for age and deepfake, while the rPPG checkpoint is approximately 27 KB. This is manageable for a dedicated service but unsuitable for a lightweight Vercel serverless function together with PyTorch and OpenCV.
- There are no latency, memory, throughput, concurrency, or hardware benchmarks in the repository. Accuracy is therefore the only currently recorded model-efficiency evidence.

Where the Project Is Lagging

1. rPPG data and evaluation are insufficient

Only 10 videos are available, and the recorded evaluation uses 4 samples. The model is trained on overlapping windows, but a reliable subject-independent test split is not established. Overlapping windows can make validation optimistic when windows from the same recording appear in different partitions. A 74.66 BPM MAE indicates the current signal extraction, temporal alignment, target construction, or model is failing badly.

Required improvement: collect substantially more subjects and recordings, split by subject before windowing, verify video/BVP timestamps, use the actual camera frame rate, and compare the learned model with a signal-processing baseline such as POS or CHROM.

2. Age estimation lacks face localization

The API feeds the entire video frame to ResNet18. Background, clothing, camera framing, and multiple people can influence the prediction. The training images are face crops, so the deployment input distribution does not match the training distribution.

Required improvement: detect faces, crop and align the largest or selected face, reject frames with poor quality, and aggregate predictions with a robust statistic such as a median or confidence-weighted mean.

3. Deepfake detection is frame based

The classifier sees independent still frames and averages fake probabilities. It does not model temporal artifacts, compression consistency, lip synchronization, eye motion, or frame-to-frame blending artifacts. A model can therefore perform well on the supplied image dataset and still fail on unseen video-generation methods.

Required improvement: create video-level train/validation/test splits, use clips rather than isolated frames, test across generators and compression levels, and add temporal modeling or a strong pretrained video backbone.

4. Dataset generalization is unproven

The recorded splits are internal to the supplied datasets. There is no cross-dataset test, demographic breakdown, device breakdown, lighting breakdown, or generator-held-out deepfake benchmark. High internal accuracy can hide shortcut learning.

Required improvement: add an external holdout set, group splits by identity/source video, report confidence intervals, and publish per-group metrics and confusion matrices.

5. Evaluation needs stronger statistical controls

The evaluation script reports point estimates but does not report confidence intervals, calibration, ROC-AUC, PR-AUC, threshold analysis, age-group error, or failure rates. The age within-five-years score should be checked because it is inconsistent with the reported MAE and may reflect an evaluation or checkpoint issue.

Required improvement: freeze a clean test set, evaluate once after model selection, add bootstrap confidence intervals, add calibration curves, and log the exact checkpoint, dataset hash, configuration, and random seed.

6. Production API safeguards are missing

CORS permits every origin. Uploaded filenames are used to construct temporary paths without explicit filename sanitization. There are no authentication, rate limiting, request size limits, MIME validation, malware scanning, structured logging, health/readiness separation, or queueing for expensive inference.

Required improvement: sanitize names with generated IDs, validate file type and duration, cap upload size, restrict CORS, add authentication and rate limits, use a job queue for long videos, and store only the minimum required data.

7. Medical interpretation is too strong

The heart-rate result is labeled Healthy, Border Line, or Needs Attention, but the rPPG model is not validated for clinical use. A webcam estimate from a short video cannot establish health status or diagnose a condition.

Required improvement: rename this to a non-diagnostic signal estimate, show confidence and signal quality, disclose limitations, and obtain clinical validation and regulatory review before presenting health claims.

8. Deployment architecture is not production ready

Vercel can serve the static frontend, but the PyTorch/OpenCV API needs a long-running or containerized inference service. The current design loads large models into one process and writes temporary files to local disk, which does not scale reliably across ephemeral serverless instances.

Required improvement: deploy the API in a container on a GPU/CPU service appropriate for inference, use object storage for uploads, move inference to background workers, add observability, and keep the frontend and API origins configurable through environment variables.

Recommended Production Roadmap

Phase 1: Make evaluation trustworthy

- Rebuild all CSV manifests from source files and verify every path.
- Remove or correct invalid age labels.
- Split age data by identity where identity metadata exists.
- Split rPPG by subject before creating windows.
- Add independent test sets and dataset version hashes.
- Re-run all metrics and investigate the age within-five-years discrepancy.

Phase 2: Improve model quality

- Add face detection, alignment, and quality filtering.
- Fine-tune stronger pretrained age and deepfake backbones with controlled augmentations.
- Train deepfake models on clips and hold out manipulation families.
- Replace or benchmark the rPPG regressor against POS/CHROM and frequency-domain methods.
- Use longer, correctly aligned rPPG windows and signal-quality gating.
- Tune deepfake thresholds on validation data rather than assuming 0.5.

Phase 3: Production engineering

- Add request validation, authentication, rate limiting, and restricted CORS.
- Add structured logs, metrics, tracing, and model/version identifiers in every response.
- Add automated tests for preprocessing, model loading, file cleanup, and API contracts.
- Containerize the API and run load tests for latency, memory, and concurrency.
- Move uploads to private object storage and use asynchronous jobs for larger videos.

- Add model monitoring for input drift, confidence drift, and human-reviewed error samples.

Phase 4: Responsible release

- Document supported populations, camera conditions, age ranges, and known failure cases.
- Remove diagnostic health wording unless clinical evidence supports it.
- Conduct fairness and subgroup analysis.
- Establish a model rollback process and an incident-response path.

Final Assessment

The project demonstrates a coherent end-to-end prototype and the deepfake image classifier is the strongest current component at 89.05% accuracy and 88.71% F1 on the recorded test split. Age estimation is usable only as a rough estimate and needs better input alignment and evaluation review. The rPPG component is not ready for production or health-related decisions: its dataset and evaluation are too small, and its measured error is extremely high.

The most valuable next investment is not a larger neural network. It is trustworthy, subject-independent data collection and evaluation, followed by face-aware preprocessing, video-level deepfake modeling, and a validated rPPG baseline. Only after those steps should production optimization and deployment scaling become the priority.